

CYBER.SOHO EDUCATIONAL HUB

Building the Invisible Attacking Lab

An anonymity-and-OPSEC lab for the Intrusion Testing course, re-engineered for a 30-student cohort on the CyberQuébec cloud range — with defender labs on student laptops

Field	Value
Course	Intrusion Testing — competency EZ06 (conduct an intrusion test); pairs with the Cyber Defense course (EZ03)
Cohort	≈ 30 students · working in 15 teams of two · up to 20 people connected to the cloud range simultaneously
Attacker infrastructure	CyberQuébec cloud range — OpenNebula 6.10 “Bubble”, FireEdge Sunstone
Defender infrastructure	Each team's own laptop — Oracle VirtualBox 7.x
Content current to	12 August 2026
Prepared by	Cyber.SoHo Educational Hub

0. What changed in this revision — read this first

Your original draft was strong on the anonymity concepts. This revision keeps that spine and fixes the things that would have broken in a shared classroom, brings every tool version and practice up to August 2026, and adds the three pieces you asked for: a provider-ready capacity request for CyberQuébec, a clean split between attacker (cloud) and defender (laptop) infrastructure, and hardware/software requirements for the students' VirtualBox machines.

The single most important correction

The original “Onion Stack” chained **Tor + double ProtonVPN out to the real Internet**. On a shared institutional cloud that faces the public Internet, that is the **wrong** architecture for a class, for three reasons:

- 1. Provider risk** — twenty teams routing live Tor exit traffic out of CyberQuébec's address space will get the range's IP ranges blocklisted and may violate the provider's acceptable-use policy. The exit-node problem becomes *the school's* problem.
- 2. Pedagogy** — students learn the *architecture* of layered anonymity just as well against a **contained target range** as against the open Internet, without the legal surface.
- 3. Reliability** — free-tier ProtonVPN throttling × 20 teams × live Tor is a support nightmare on demo day.

So this revision reframes the lab into two clean halves, which is also exactly the attacker-versus-defender split your course needs:

	ATTACKER side	DEFENDER side
Runs on	CyberQuébec cloud range (OpenNebula/Sunstone)	The team's own laptop (VirtualBox)
Who builds it	The provider builds the range; students instantiate their pod	Each team builds it themselves — it is graded
What it contains	The anonymity stack (OPNsense + Whonix + Kali) and the target machines	A small defended network to attack and to harden
Internet exposure	None — the range is self-contained; “the Internet” is simulated inside it	None — fully offline on the laptop

	ATTACKER side	DEFENDER side
The lesson	How a professional stays un-attributable, and how the layers fail	What those attacks look like from the blue-team seat

Everything the anonymity chain teaches is preserved

The Onion-Stack concept, the “each node knows only its neighbours” principle, the four leak vectors (DNS, IPv6, WebRTC, kill-switch), and the full verification battery are all still here. What changes is the exit: the chain now terminates at a simulated-Internet gateway and target range inside CyberQuébec, not at a real Tor exit node on the public Internet. Students still build and verify the entire stack; they simply attack a lawful target that lives one hop away instead of across the world.

0.1 Other updates to 2026 best practice

Area	Original draft	This revision (current to Aug 2026)
Firewall/router	OPNsense (unversioned)	OPNsense 25.7 “Visionary Viper” LTS branch (or 26.1 CE). WireGuard is now in-core — no plugin.
VPN protocol	WireGuard vs OpenVPN discussed	WireGuard only for the lab: kernel-fast, smaller attack surface, in-core in OPNsense.
Anonymity OS	Whonix (VirtualBox OVA)	Whonix 17 for VirtualBox — supported through 2026 by Power Up Privacy. Import unchanged.
“Double VPN” exit	ProtonVPN Free x2 → real Internet	Replaced by two internal VPN hops inside the range → simulated-Internet gateway. No real-world exit.
Attacker workstation	Whonix-Workstation, Kali optional	Kali 2026.x as the working box behind the Gateway; Whonix-WS kept as the “no-leak” reference.
Kill-switch proof	Wireshark on WAN	Same, plus an explicit OPNsense floating-rule tag and a scripted verification checklist.
Professional note	Mullvad (cash/crypto)	Still the real-world recommendation — taught as theory only; the class never pays for or uses it.

Note on competency alignment. This lab belongs to the *Intrusion Testing* course (EZ06), which is a different course from the Cyber Defense course (EZ03) your Day-01 materials serve. The defender half below is deliberately written so it can double as a bridge exercise between the two.

1. One-hour lecture — “Building the Invisible Attacking Lab”

Delivery: 60 minutes. Voice is beginner-friendly; every abbreviation is expanded in round brackets on first use. This is theory only — no machine is touched until the lab.

A. Why attacker OPSEC matters (10 min)

- A professional penetration tester works from clean, un-attributable infrastructure — never from a home Internet connection. OPSEC means Operational Security: keeping your own identity out of the target's records.
- Three surfaces expose a tester: the Internet Service Provider's (ISP) connection logs, a Virtual Private Network (VPN) provider's logs, and surveillance of Tor (The Onion Router) exit nodes.
- The legal frame: the Rules of Engagement (RoE) require that **only the tester's authorised identity** appears in the target's logs — never personal attribution, and never a bystander's address.
- Real incidents where OPSEC failed did so through **DNS (Domain Name System) leaks**, **WebRTC (Web Real-Time Communication) browser leaks**, and **IPv6 (Internet Protocol version 6) bypass** — the same four failures this lab makes you test for.
- Professional equivalent (theory only): in a real engagement, a no-log provider such as [Mullvad](#) — paid with cash or Monero, no email required — replaces a free VPN. *We never use or pay for it in class; you learn why it exists.*

Ethical boundary — repeat it every session

This entire stack is built to attack authorised targets only, under the same Rules of Engagement you signed in Week 1. Anonymity is a professional-safety and client-confidentiality tool, not a shield for unauthorised activity. Unauthorised access is a criminal offence under the Criminal Code of Canada s.342.1, the U.S. Computer Fraud and Abuse Act (18 U.S.C. §1030), EU Directive 2013/40/EU and Spain's Ley Orgánica 10/1995. In this course, your only lawful target is the range machine assigned to you.

B. The layered anonymity architecture (15 min)

The core idea is the **Onion Stack**: each layer hides the one beneath it from the next hop, so no single point on the path can see both who you are and what you are doing.

Classroom chain (contained inside the CyberQuébec range):

```
Kali / Whonix-Workstation      (the attacker's keyboard)
-> Whonix-Gateway             (Tor: three encrypted relays)
-> OPNsense (VPN hop #2)      (WireGuard, 'exit' side)
-> OPNsense (VPN hop #1)      (WireGuard, 'entry' side)
-> Range edge / sim-Internet gateway
-> Target machine             (your authorised lab target)
```

Original real-world chain (taught as contrast, not built): the same order but ending *-> Host -> ISP -> Internet -> Target*. The only change for the classroom is the exit.

What each layer can see:

This node...	...can only see
The target	A range-edge / exit address — never the student
The Tor exit relay	Traffic arriving from VPN hop #2
VPN hop #2	Traffic from VPN hop #1

This node...	...can only see
VPN hop #1	An encrypted tunnel from OPNsense — no content, no destination
The range fabric	Encrypted tunnels only

The one principle to memorise

Each node knows only its immediate predecessor and successor. No single node — and no single subpoena to one provider — can reconstruct the full chain. That is the whole point of layering, and it is why breaking one layer is not enough to deanonymise a professional.

C. OPNsense — the virtual firewall/router (10 min)

- [OPNsense](#) is an open-source, FreeBSD-based firewall — the professional-grade sibling of pfSense. This lab uses the **25.7 “Visionary Viper” LTS** branch (or 26.1 Community Edition).
- Role here: it is the LAN gateway for the pod's virtual machines, it enforces the VPN routing, and it implements the kill switch at the router level.
- Kill-switch logic: if a VPN tunnel drops, OPNsense **blocks all upstream traffic** so nothing escapes in cleartext. Zero leak, by design.
- Leak-prevention features configured in OPNsense:
 - force all DNS through the built-in [Unbound](#) resolver with DNS-over-TLS — no plaintext DNS leaves the pod;
 - IPv6 fully disabled — Tor and the lab VPNs are IPv4-only, and IPv6 is the classic tunnel-bypass;
 - outbound rules tag lab traffic so anything not bound for the VPN gateway is dropped if the tunnel is down.
- Why OPNsense rather than “just run a VPN inside Kali”: a VPN on one VM protects *only that VM*; OPNsense protects **the whole pod** at the router, which is how real networks are built. WireGuard is now in-core — no plugin to install.

D. The two internal VPN hops (10 min)

- “Double VPN” means chaining two separate encrypted tunnels through two different points, so neither hop sees both ends of the conversation.
- In the *original* design these were two free ProtonVPN accounts in two countries, routing to the real Internet. In **this** design both hops live **inside the range** and terminate at a simulated-Internet gateway — the concept is identical, the public-Internet exposure is gone.
- Protocol: **WireGuard** only — faster and a smaller attack surface than OpenVPN, and kernel-native.
- Trade-off students must feel: every hop adds latency (~200–400 ms in the real world). This matters when timing-sensitive tools run — e.g. [nmap](#) timing templates — so the lab has them measure it.
- Real-world note (theory only): here a no-log provider such as Mullvad would replace the second hop. The classroom **never** pays for or connects to a commercial VPN.

E. Whonix — the isolation engine (10 min)

- [Whonix](#) is two virtual machines with a strict separation of duties (Whonix 17, supported through 2026):
 - **Whonix-Gateway** — the only VM that touches the network; it routes *all* traffic through Tor. If it is compromised it reveals only the VPN-hop address, never the real one.
 - **Whonix-Workstation** — has **no direct network access**; it sees only the Gateway's internal address ([10.152.152.10](#)). Even malware running on it cannot leak the real address.

- Key property: the Workstation has no DNS resolver of its own, so DNS **cannot** leak — everything goes Gateway → Tor.
- Tor path = three relays: **Guard, Middle, Exit** — each knows only the adjacent hops.
- In this lab the Workstation is **replaced or augmented by Kali** on the Whonix internal network, so students attack from a full toolkit while keeping the no-leak property.
- Whonix is **not** a silver bullet: a Tor exit relay sees cleartext if the tester does not use HTTPS/TLS. **Always use HTTPS/TLS to the target.**

F. Threat model & leak vectors (10 min)

The four leaks every student must be able to name, cause, and prove are closed:

1. **DNS leak** — the real resolver bypasses the tunnel and the ISP sees your queries.
2. **IPv6 leak** — IPv6 traffic slips past an IPv4-only tunnel.
3. **WebRTC leak** — a browser interface reveals the local/public address even through a VPN.
4. **Kill-switch failure** — the VPN drops for a moment and cleartext escapes.

Verification tools used in the lab: ipleak.net, dnsleaktest.com (or their in-range mirrors), the browser's own WebRTC test page, and **Wireshark** on the OPNsense upstream interface.

Practical OPSEC rules for students:

- Never sign into a personal account from the Whonix Workstation or Kali.
- Never reuse a tool signature (same browser User-Agent, same nmap timing profile) across engagements.
- Snapshot every VM before and after each session; revert after use.
- Treat the Workstation as **disposable** — rebuild it from a clean snapshot after each major exercise.

G. The real-world professional stack (5 min)

Layer	Classroom (free, contained)	Professional real world
Firewall/router	OPNsense VM (25.7 LTS)	OPNsense on dedicated hardware, or a hardened VPS
VPN #1	In-range WireGuard hop	Mullvad (cash/Monero, WireGuard)
VPN #2	In-range WireGuard hop	Second Mullvad account or a different provider
Anonymisation	Whonix + Tor	Whonix + Tor, or I2P (Invisible Internet Project)
Workstation	Kali on Whonix internal net	Kali on Whonix internal net
Exit	Simulated-Internet gateway (in range)	Real Tor exit — with all its legal weight
Payment	None — nothing is purchased	Monero or cash only

2. Capacity request to CyberQuébec — step-by-step

This is the section to send to the CyberQuébec team. It gives them the exact shape of the range to build so that **up to 20 people can work simultaneously**, the per-pod specification, and the platform-correct way to wire and isolate it. It follows CyberQuébec's own conventions: virtual machines are attached to networks through **Service Templates (roles)**, not by hand-attaching network cards, and isolation uses the “**Virtual Lan - OpenVSwitch Network**” (VXLAN) template.

Consult the platform's own documentation alongside this request

CyberQuébec publishes procedures in BookStack at documentation.cyberquebec.org — in particular the “Guide de l'enseignant” shelf and the “Gestion de laboratoire” book. Where this request and the BookStack differ on a UI detail, follow the BookStack; the platform is the source of truth for its own screens.

2.1 What we need, in one paragraph for the provider

Plain-language summary for the CyberQuébec team

We are a class of **~30 students in 15 teams of two**, and we need up to **20 people connected at once**. Please provision an **Intrusion-Testing range** as **15 identical, isolated pods** (one per team) from a single OneFlow Service Template, so each team gets its own private copy of the anonymity stack plus a small target range. Each pod is four small VMs on one isolated VXLAN segment. The range must be **self-contained — no route to the public Internet**; a lightweight “sim-Internet” gateway inside each pod plays the role of the outside world. Sizing, roles, and network isolation are specified below.

2.2 Per-pod specification (one team of two)

Each pod is one Service Template made of four VM roles on one isolated segment:

Role (VM)	Base image	vCPU	RAM	Disk	Purpose in the pod
opnsense-edge	OPNsense 25.7	2	2 GB	16 GB	Router/firewall; both WireGuard hops; kill switch; Unbound DNS
whonix-gateway	Whonix-Gateway 17	1	1.5 GB	10 GB	Tor entry for the pod (3 relays)
kali-attacker	Kali 2026.x	2	3 GB	30 GB	The student keyboard, behind the Gateway
sim-inet + target	Debian 12 (minimal)	1	1.5 GB	20 GB	Simulated-Internet gateway + one authorised target service
PER POD TOTAL	—	6	8 GB	76 GB	Four VMs, one VXLAN segment

Optional Whonix-Workstation. If you want the pure “no-leak reference” VM in addition to Kali, add a fifth role (1 vCPU / 1 GB / 10 GB). It is not required — Kali on the Whonix internal network already gives the anonymity property — so it is omitted from the totals to keep the range light.

2.3 Fleet sizing — what to actually build

Scenario	Pods live	vCPU	RAM	Disk	Notes
All 15 teams at once	15	90	120 GB	1.14 TB	Full cohort; safe headroom target
20 people = 10 teams live	10	60	80 GB	760 GB	Your stated simultaneity floor
Recommended host budget	15 + spare	~110	~160 GB	~1.5 TB	Includes overcommit + snapshots

Scenario	Pods live	vCPU	RAM	Disk	Notes
Provider planning notes CPU overcommit: these VMs are I/O- and idle-heavy, not CPU-bound. A 2–3× vCPU overcommit is safe, so ~90 vCPU of guests fits comfortably on ~32–40 physical cores. RAM is the real limit: do not overcommit memory for Whonix/Kali. Provision the full ~120 GB for 15 live pods, ~160 GB with headroom. Disk: please use linked clones / thin provisioning from the golden images — 15 full copies of Kali would be wasteful; thin-provisioned pods share the backing store. Concurrency: 20 simultaneous console sessions via Sunstone's Guacamole/VNC is well within range; if bandwidth is tight, students can also SSH/RDP into their Kali directly.					

2.4 Step-by-step build request for the CyberQuébec administrator

These steps use CyberQuébec's platform conventions. The teaching team does not need admin rights to run them — this is the request for the provider to action, or to grant the instructor an org with quota to do so.

Step 1 — Create the class organisation and quota

1. In Sunstone (as administrator), create a **Group/organisation** named e.g. `INTRUSION-2026`, assigned the **cloud view**.
2. Set the group **quota** to the recommended host budget in §2.3 (~110 vCPU, ~160 GB RAM, ~1.5 TB) so pods cannot exhaust the range.
3. Create **15 users** (one per team), e.g. `team01` ... `team15`, each in that group. Give each team its own login so pods stay isolated per team.

Step 2 — Register the four golden images

4. Under **Storage → Images**, register the four base images: OPNsense 25.7, Whonix-Gateway 17, Kali 2026.x, Debian 12 minimal. Mark them **persistent = No** so each instantiation gets a fresh non-persistent clone.
5. If CyberQuébec already ships a base image, note its credentials: the platform base login is `env-admin` / `admin-env`. Confirm which images exist before uploading duplicates.

Step 3 — Build the isolated pod network (VXLAN)

6. Under **Network → Virtual Networks**, create the pod segment from the “**Virtual Lan - OpenVSwitch Network**” **template** — this is the VXLAN type CyberQuébec uses for isolation.
7. Give it a private range, e.g. `192.168.10.0/24`, with **no default route to the Internet**. The pod's OPNsense is the only gateway; the sim-Internet VM plays “outside”.
8. Because this is a per-team template, the VXLAN VNI is allocated per instantiation — so each of the 15 pods is on its *own* isolated segment automatically. Teams cannot see each other's traffic.

Why Service Templates, not hand-attached NICs

On CyberQuébec, VMs are wired to networks through the **Service Template roles**, not by manually attaching a network card to each VM. Define the wiring once in the template; every instantiated pod is wired identically and correctly. This is the platform's intended workflow and the reason 15 identical pods are trivial to spin up.

Step 4 — Create the four VM Templates (roles)

9. Under **Templates** → **VM Templates**, create one template per role using the sizes in §2.2. Attach each to the pod VXLAN.
10. Wire the roles so the traffic path is: `kali` → `whonix-gateway` → `opnsense` (`wg2` → `wg1`) → `sim-inet/target`. The Kali and Gateway sit on the Whonix internal side; OPNsense bridges internal to the sim-Internet side.
11. On OPNsense, pre-load a config (via contextualisation) that already has: both WireGuard instances `wg1/wg2`, Unbound with DNS-over-TLS, IPv6 disabled, and the kill-switch floating rule. This saves 45 minutes of per-team setup and makes the lab about **verifying** the stack, not building the router from scratch.

Step 5 — Assemble the OneFlow Service Template (the pod)

12. Under **Templates** → **Services** (OneFlow), create a Service Template named e.g. `intrusion-pod` containing the four VM roles as a single deployable unit.
13. Set role start order so **OPNsense boots first**, then the Gateway (Tor bootstraps), then Kali, then the target.
14. Share the Service Template with the INTRUSION-2026 group so every team can instantiate **their own** copy with one click.

Step 6 — Hand-off to teams

15. Each team logs in and instantiates **one** `intrusion-pod` service. That gives them their private four-VM anonymity range.
16. Teams reach their Kali via the Sunstone **Guacamole/VNC console**, or by SSH/RDP if the provider exposes a jump path. Twenty simultaneous consoles are within the platform's capacity.
17. At end of session, teams **terminate** (or the instructor schedules automatic termination) so the range reclaims capacity for the next class.

One-line message you can paste to CyberQuébec

“Please build an Intrusion-Testing range as a single OneFlow Service Template (‘intrusion-pod’) of four small VM roles — OPNsense 25.7, Whonix-Gateway 17, Kali 2026.x, Debian-12 target — on one isolated ‘Virtual Lan - OpenVSwitch Network’ (VXLAN) segment per instantiation, self-contained with no public-Internet route, shared to a 15-user ‘INTRUSION-2026’ group with a ~110 vCPU / 160 GB RAM / 1.5 TB quota, so up to 20 students in 15 teams can each run their own pod simultaneously. Thin-provision from golden images.”

3. The attacker lab on the CyberQuébec range — student steps

Once the provider has published the `intrusion-pod` service, each team runs this. Because the router comes pre-configured (Step 4.3 above), the students' job is to **bring the stack up in order, then prove every layer and every leak-vector** — which is exactly the skill being assessed. Estimated 3 hours.

Phase 1 — Instantiate and map the pod (20 min)

1. Log in to Sunstone and instantiate one `intrusion-pod` service. Wait for all four VMs to reach RUNNING.
2. Before touching anything, draw the traffic-flow diagram in your lab notebook: Kali → Gateway → OPNsense (wg2→wg1) → sim-Internet → target, with the address of each hop. You submit this diagram as evidence.
3. Open the OPNsense console from Sunstone; confirm it booted and both WireGuard instances show a recent handshake.

Phase 2 — Verify the router and kill switch (30 min)

1. In the OPNsense web interface: confirm `Unbound` is enabled, DNS-over-TLS is on, and IPv6 is disabled on all interfaces.
2. Confirm the kill-switch floating rule exists: outbound upstream traffic not tagged for the WireGuard gateway is blocked.
3. Kill-switch test: administratively stop `wg1`. From Kali, confirm all traffic stops. Capture the OPNsense upstream interface in Wireshark and confirm no cleartext escapes. Restart `wg1`; traffic resumes. Screenshot both states.

Phase 3 — Bring up Whonix and Kali (40 min)

1. Start Whonix-Gateway; complete first-run; confirm Tor bootstraps to 100%. Confirm the Tor Guard relay address is a **VPN-hop address, not the pod edge**.
2. Confirm Kali is on the Whonix internal network (`10.152.152.0/24`) with the Gateway as its only route: `10.152.152.10`.
3. From Kali: `curl https://check.torproject.org` (or the in-range mirror) confirms you are routed through Tor. Install `proxychains4` as a second SOCKS5 layer via the Gateway (`127.0.0.1:9050`).

Phase 4 — Full-chain and leak verification (40 min)

Run the full battery from Kali (and the Whonix-Workstation if you deployed it). Every result is evidence.

Test	Command / page	Pass condition
IP leak	<code>curl https://ipleak.net/json/</code>	Shows only the exit/sim-Internet address — never the pod edge or a student address
DNS leak	<code>dnsleaktest</code> (extended)	All resolvers are Tor/Unbound — never a bare ISP resolver
IPv6 leak	<code>curl -6 https://ipv6.icanhazip.com</code>	Times out — IPv6 is blocked
WebRTC leak	browser WebRTC test page	Local address is 10.152.152.x only — never a host address
Kill switch	stop <code>wg1</code> , watch Wireshark	All traffic halts; nothing on the upstream interface
Anonymised scan	<code>proxychains4 nmap -sT -Pn <target></code>	Scan completes through the chain against the authorised target only

Phase 5 — Snapshot & report (20 min)

1. Snapshot all VMs in the verified-clean state (name: **CLEAN-VERIFIED-<date>**).
2. Submit: the annotated network diagram, one screenshot per leak test (pass/fail), the kill-switch result, and the verified exit address versus your real address.
3. Reflection: “At which layer would an investigator most likely attempt to deanonymise an attacker using this stack, and why?” — tie back to the Week-1 legal and ethics framework.

4. The defender lab on the student laptop (VirtualBox)

The attacker range lives in the cloud; the **defenders' lab lives on each team's own laptop**, in VirtualBox, fully offline. This is where students see what the attacks from Section 3 look like from the blue-team seat, and where they build a small network to harden. It is graded.

Why the defender side is on the laptop, not the cloud

Two reasons. First, it keeps the cloud range light — the provider only has to host the attacker pods. Second, blue-team work means owning the whole machine: capturing packets, breaking things, reverting, and rebuilding — which is exactly what a personal VirtualBox on your own laptop is best for, and what most of you already use.

4.1 Laptop hardware requirements

Component	Minimum (works)	Recommended (comfortable)
CPU	4 cores (Intel VT-x / AMD-V enabled in BIOS/UEFI)	6–8 cores with virtualisation on
RAM	16 GB	32 GB — the single biggest comfort factor
Free disk	120 GB free (SSD)	200 GB free on NVMe SSD
OS	Windows 10/11 64-bit, macOS 12+ (Intel), or Linux	Same, fully updated
Other	Virtualisation enabled in firmware; laptop on AC power	Second monitor helps for multi-VM work

The virtualisation switch — the #1 support ticket

If VirtualBox shows only 32-bit guest options, or a VM refuses to start with a VT-x error, virtualisation is turned off in firmware. Reboot into BIOS/UEFI and enable **Intel VT-x / VT-d** or **AMD-V / SVM Mode**. On Windows, also disable Hyper-V / “Memory Integrity” if VirtualBox and Hyper-V conflict. Do this *before* the first lab.

4.2 Laptop software requirements

Software	Version	Notes
Oracle VirtualBox	7.1.x (7.0+ minimum)	Free. Most of the cohort already uses it.
VirtualBox Extension Pack	Match the VirtualBox version exactly	For USB/RDP; PUEL licence — free for personal/educational use.
OPNsense ISO	25.7 LTS (dvd, amd64)	Edge firewall for the defended network.
A Windows target	Windows 10/11 Evaluation VM	Free 90-day evaluation image from Microsoft.
A Linux target	Debian 12 or Ubuntu Server 24.04 LTS	The second host to harden.
Kali (optional, light)	Kali 2026.x (2 GB RAM)	Only if attacking locally as well; the cloud pod is the main attacker.
Wireshark	Latest (bundled in Kali; installable on host)	To watch the attacks arrive.
7-Zip / disk tool	Latest	To unpack images.

Storage reality check

OPNsense (16 GB) + a Windows VM (40–50 GB) + a Linux VM (20 GB) + optional Kali (30 GB) ≈ 110–120 GB. That is why 120 GB free is the floor. Use dynamically-allocated disks and delete snapshots you no longer need. If a laptop is tight on RAM, run the Windows target and Kali one at a time rather than together.

4.3 The defender network on the laptop

Create three VirtualBox networks in the **Host Network Manager**, then wire the VMs. Everything is **Internal Network** or **Host-Only** — nothing bridges to the real world.

VirtualBox network	Type	Carries
def-wan	Internal Network	Between OPNsense WAN and the (simulated) outside — or NAT if you need updates, then disable
def-lan	Internal Network	The defended LAN: Windows + Linux targets sit here behind OPNsense
def-mgmt	Host-Only	Management access from the host to the OPNsense web interface

VM wiring:

- `opnsense-edge` — Adapter 1 → `def-wan`; Adapter 2 → `def-lan` (LAN 192.168.20.1/24); Adapter 3 → `def-mgmt`.
- `win-target` and `linux-target` — one adapter each → `def-lan`, addresses from OPNsense DHCP.
- Set **promiscuous mode = Deny** on all adapters except where you are deliberately capturing, and confirm IPv6 is disabled on the host adapters — mirroring the attacker-side discipline.

4.4 What the defenders actually do

1. Harden both targets to the Day-01 baseline: host firewall on, unused services off, patched, least-privilege accounts, logging forwarded. (Re-use `r1e04_Get-DefenseBaseline.ps1` from the Cyber Defense course on the Windows target.)
2. Stand up detection: at minimum, Wireshark on `def-lan`; ideally a small Suricata or Zeek sensor, so the scan from Section 3 is visible.
3. Have the paired attacker (from the cloud pod, or a local Kali) run `nmap` and one authorised exploit attempt against a target; the defender captures it, identifies it, and writes the detection up.
4. Produce the four lab artifacts (working config, screenshot evidence, test result, written finding) — the same evidence discipline as every other lab in the programme.

The bridge to the Cyber Defense course

This defender half is deliberately compatible with the Cyber Defense (EZ03) materials: the seven-layer model, the CIS-IG1 hygiene list, the evidence register, and the residual-risk reasoning all transfer directly. A student who does both courses sees the same network from both chairs — which is the whole point.

5. Placement, prerequisites, and pre-flight checklist

5.1 Where this fits

Run this as **Week 1, Lab 2** (immediately after initial lab setup), or as a standalone **Week 2 Class 1** warm-up, so that every subsequent lab in Weeks 2–12 is conducted from inside this contained stack. That gives students authentic OPSEC discipline from day one — while keeping the whole exercise lawful and inside CyberQuébec.

5.2 Before the class — provider side

- CyberQuébec has published the intrusion-pod Service Template to the INTRUSION-2026 group with the \$2.3 quota.
- The four golden images are registered and thin-provisioning is enabled.
- A test instantiation of one pod has been verified end-to-end by the instructor.

5.3 Before the class — student side

- VirtualBox 7.1.x + matching Extension Pack installed; virtualisation enabled in firmware.
- 16 GB RAM confirmed; 120 GB free disk confirmed; laptop will be on AC power.
- OPNsense 25.7 ISO, a Windows evaluation VM, and a Debian/Ubuntu server image downloaded.
- Sunstone login received from the instructor and tested.

6. Disclaimer, intellectual property, and lawful-use notice

Ownership. This document and its structure are the intellectual property of Cyber.SoHo Educational Hub, protected by the Copyright Act (R.S.C., 1985, c. C-42) of Canada and international convention. © 2026 Cyber.SoHo Educational Hub. All rights reserved. Enrolled students may use and adapt it for the assessed work of this course only.

No affiliation. Cyber.SoHo Educational Hub is independent and is not affiliated with, sponsored by, or endorsed by CyberQuébec, OpenNebula, Whonix, the Tor Project, OPNsense/Deciso, Kali/OffSec, Oracle, Mullvad, Proton, or any other product, project or platform named. All trademarks belong to their owners and are used for identification and education only. Platform procedures are summarised for teaching; the provider's own BookStack documentation prevails over anything written here.

No warranty. Provided “as is” for education only. Not professional, legal or security-consulting advice. Versions and behaviours accurate on 12 August 2026 may change; re-verify against primary sources. Verify any external link with [virustotal.com](https://www.virustotal.com) before opening it.

Lawful and authorised use only — the boundary of this entire lab

Every technique, tool and configuration here is to be used only on systems you own or are explicitly authorised in writing to test — in this course, only the CyberQuébec range machine assigned to you, or the VMs on your own laptop. Anonymity infrastructure is a professional-safety and client-confidentiality tool; it is never a shield for unauthorised activity. Unauthorised access to, interference with, or interception of a computer system or its data is a criminal offence, including under the Criminal Code of Canada s.342.1 and s.430(1.1), the U.S. Computer Fraud and Abuse Act (18 U.S.C. §1030), EU Directive 2013/40/EU, and Spain's Ley Orgánica 10/1995. Quebec's Law 25, Canada's PIPEDA and the EU GDPR may apply to any personal data encountered. If you are unsure whether an action is authorised, it is not — stop, document, and ask in writing. “Hacking back” is prohibited and unlawful in Canada regardless of provocation.

Cyber.SoHo Educational Hub

From the Industry to the Classroom — Building the IT's Future, Today and Together !

© 2026 — ALL Rights Reserved